图像黑帽运算

图像黑帽运算
- 例程效果
- 原理讲解
  - 什么是图像的黑帽运算
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置黑帽运算参数
    - 图像处理与黑帽运算
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/16.rgb888_blackhat.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的图像黑帽运算。



原理讲解

什么是图像的黑帽运算

黑帽运算（Black-Hat Operation）是一种形态学图像处理方法，用于突出图像中的暗细节或小区域，特别是那些比周围区域更暗的部分。黑帽运算的结果是闭运算（Closing）与原图像之间的差值，数学上可以表示为：

黑帽 = 闭运算结果 - 原图像

- **闭运算（Closing）**：闭运算是形态学操作中的一种，先对图像进行膨胀（Dilation），再进行腐蚀（Erosion）。这种操作可以填充图像中的小孔或连接断开的区域，同时保持整体形状。
- **黑帽运算的效果**：通过将闭运算结果与原图像相减，黑帽运算能够提取出在闭运算中被“填充”或“忽略”的暗区域细节。这些区域通常是图像中较小的暗色结构或纹理。

黑帽运算在图像处理中的应用包括：

- 检测图像中的暗细节或小缺陷。

- 增强图像中的暗纹理或边缘。
- 在目标检测中用于分离小尺寸的暗目标。

在实际应用中，黑帽运算的参数（如卷积核大小和迭代次数）会影响检测的细节范围和效果，而二值化阈值则决定了结果图像中哪些区域被认为是“暗区域”。

代码概述

导入模块

```
import time, os, gc
from machine import Pin
from media.sensor import *    # 摄像头接口 / Camera interface
from media.display import *   # 显示接口 / Display interface
from media.media import *     # 媒体资源管理器 / Media manager
import cv_lite                # cv_lite 扩展模块（含黑帽操作）/ AI CV extension
(Black-Hat function)
import ulab.numpy as np       # NumPy-like ndarray for MicroPython
```

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(id=2, width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)  # 设置像素格式为 RGB888 / Set pixel format to
RGB888
```

- 初始化摄像头，设置分辨率为 1280x960，帧率为 90 FPS。
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合处理彩色图像的黑帽运算）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0],
to_ide=True)
```

初始化显示模块，使用 ST7701 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置黑帽运算参数

```
kernel_size = 3          # 卷积核尺寸（建议为奇数）/ Kernel size (recommended odd)
iterations = 1           # 运算迭代次数 / Number of morphological passes
threshold_value = 100    # 二值化阈值（0=使用 Otsu 自动阈值）/ Threshold for
binarization (0 = Otsu)
clock = time.clock()     # 启动帧率计时器 / Start FPS timer
```

- `kernel_size`：卷积核尺寸，用于形态学操作（膨胀和腐蚀），建议为奇数，值越大处理范围越大，影响检测细节的大小。
- `iterations`：运算迭代次数，表示形态学操作的重复次数，值越大效果越显著，但计算量增加。
- `threshold_value`：二值化阈值，用于将处理后的图像转换为二值图像，值为 0 时使用 Otsu 自动阈值算法。
- `clock`：用于计算帧率。

图像处理与黑帽运算

```
while True:
    clock.tick()  # 开始计时 / Start frame timing

    # 获取一帧图像并转换为 ndarray / Capture a frame and convert to ndarray
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()

    # 执行黑帽运算 / Apply Black-Hat operation
    # 黑帽 = 闭运算 - 原图 / Black-Hat = Closing - Original
    result_np = cv_lite.rgb888_blackhat(
        image_shape,
        img_np,
        kernel_size,
        iterations,
        threshold_value
    )

    # 构造图像对象并显示 / Wrap result as image and display
    img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                          alloc=image.ALLOC_REF, data=result_np)
    Display.show_image(img_out)

    # 清理内存并打印帧率 / Cleanup and print FPS
    gc.collect()
    print("blackhat fps:", clock.fps())
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **黑帽运算**：调用 `cv_lite.rgb888_blackhat()` 函数进行黑帽运算，返回处理后的图像 NumPy 数组。
- **图像包装和显示**：将处理结果包装为灰度图像对象，并显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()                          # 停止摄像头 / Stop sensor
Display.deinit()                       # 关闭显示输出 / Deinit display
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)  # 设置退出点 / Safe exit point
time.sleep_ms(100)                     # 稍作延迟 / Short delay
MediaManager.deinit()                  # 释放媒体资源 / Release media manager
```

参数调整说明

- `kernel_size`：卷积核尺寸，值越大处理的区域范围越大，适合检测较大的暗细节；建议使用奇数（如 3、5、7），从 3 开始测试。
- `iterations`：运算迭代次数，值越大形态学操作效果越明显，但计算量增加；建议从 1 开始，根据需求逐步增加。
- `threshold_value`：二值化阈值，值越高需要更亮的像素才能被保留，值为 0 时使用 Otsu 自动阈值；建议根据图像亮度从 50 到 150 测试，或设置为 0 使用自动阈值。